

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Comparative Analysis (Low)

Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze

7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze
8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze
9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I B Aishwarya/192421432(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B Aishwarya

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehend comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation ; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1Q) Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.

ANS

File organization refers to the way records are physically stored in a file or database table. Two common methods are Sequential File Organization and Heap File Organization.

1.Sequential File Organization

In sequential file organization, records are stored in a particular order, usually according to a search key such as Student_ID, Employee_ID, or Account_No.

Example:

Student_ID	Name	Department
101	Arun	IT
102	Ravi	CSE
103	Priya	ECE
104	Kiran	IT

The records are maintained in increasing order of Student_ID.

Insertion:

Insertion is comparatively slower because the new record must be placed at the correct position. If there is no free space, records may need to be shifted or an overflow area may be used.

Deletion:

Deletion can be slower because the record has to be located and the sequential organization may need to be maintained. Deleted space may also create gaps.

Retrieval:

Retrieval is efficient, especially for sequential access and range queries, because records are already ordered. Binary search can also be used when the file is suitable for it.

Heap File Organization

In heap file organization, records are stored without any particular order. A new record is generally placed wherever free space is available, often at the end of the file.

Example:

Student_ID	Name	Department
103	Priya	ECE
101	Arun	IT
104	Kiran	IT
102	Ravi	CSE

There is no requirement that Student_ID be sorted.

Insertion:

Insertion is very fast because the system can simply place the new record in available free space or at the end of the file.

Deletion:

Deletion is relatively simple after locating the record. The space can be marked as free and reused later.

Retrieval:

Searching for a particular record can be slow because there is no ordering. A sequential scan may be required unless an index exists.

2. Comparison

Operation	Sequential File	Heap File
Insertion	Slower	Faster
Deletion	Comparatively slower	Faster
Searching	Faster if ordered/search key is used	Slower without an index
Sequential retrieval	Very efficient	Less efficient
Range queries	Efficient	Less efficient
Storage order	Sorted/ordered	Unordered
Best suited for	Reports, range searches, ordered processing	Frequent insertions and general-purpose storage

Conclusion

Sequential file organization is preferable when retrieval and ordered/range access are important.

Heap file organization is preferable when fast insertion and simple deletion are more important.

In short:

Sequential File → Better retrieval, slower insertion/deletion

Heap File → Faster insertion/deletion, slower searching

```
mysql>
mysql> SELECT * FROM HeapFile;
+-----+-----+-----+
| Record_ID | Student_Name | Department |
+-----+-----+-----+
| 101 | Ravi | CSE |
| 102 | Priya | ECE |
| 103 | Arun | IT |
| 104 | Meena | CSE |
| 105 | Kiran | ECE |
| 106 | Rahul | IT |
| 107 | Sneha | CSE |
+-----+-----+-----+
7 rows in set (0.00 sec)

mysql> SELECT *
-> FROM SequentialFile
-> WHERE Record_ID = 105;
+-----+-----+-----+
| Record_ID | Student_Name | Department |
+-----+-----+-----+
| 105 | Kiran | ECE |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT *
-> FROM HeapFile
-> WHERE Record_ID = 105;
+-----+-----+-----+
| Record_ID | Student_Name | Department |
+-----+-----+-----+
| 105 | Kiran | ECE |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> DELETE FROM SequentialFile
-> WHERE Record_ID = 103;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> SELECT * FROM SequentialFile ORDER BY Record_ID;
+-----+-----+-----+
| Record_ID | Student_Name | Department |
+-----+-----+-----+
| 101 | Ravi | CSE |
| 102 | Priya | ECE |
| 104 | Meena | CSE |
| 105 | Kiran | ECE |
| 106 | Rahul | IT |
| 107 | Sneha | CSE |
+-----+-----+-----+
6 rows in set (0.00 sec)

mysql> DELETE FROM HeapFile
-> WHERE Record_ID = 103;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> SELECT * FROM HeapFile;
+-----+-----+-----+
| Record_ID | Student_Name | Department |
+-----+-----+-----+
| 101 | Ravi | CSE |
| 102 | Priya | ECE |
| 104 | Meena | CSE |
| 105 | Kiran | ECE |
| 106 | Rahul | IT |
| 107 | Sneha | CSE |
+-----+-----+-----+
6 rows in set (0.00 sec)

mysql> INSERT INTO SequentialFile VALUES
-> (108, 'Kavya', 'CSE');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO HeapFile VALUES
-> (108, 'Kavya', 'CSE');
Query OK, 1 row affected (0.02 sec)
```

```

mysql>
mysql> SELECT * FROM HeapFile;
+-----+-----+-----+
| Record_ID | Student_Name | Department |
+-----+-----+-----+
| 101 | Ravi | CSE |
| 102 | Priya | ECE |
| 103 | Arun | IT |
| 104 | Meena | CSE |
| 105 | Kiran | ECE |
| 106 | Rahul | IT |
| 107 | Sneha | CSE |
+-----+-----+-----+
7 rows in set (0.00 sec)

mysql> SELECT *
-> FROM SequentialFile
-> WHERE Record_ID = 105;
+-----+-----+-----+
| Record_ID | Student_Name | Department |
+-----+-----+-----+
| 105 | Kiran | ECE |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT *
-> FROM HeapFile
-> WHERE Record_ID = 105;
+-----+-----+-----+
| Record_ID | Student_Name | Department |
+-----+-----+-----+
| 105 | Kiran | ECE |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> DELETE FROM SequentialFile
-> WHERE Record_ID = 103;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> SELECT * FROM SequentialFile ORDER BY Record_ID;
+-----+-----+-----+
| Record_ID | Student_Name | Department |
+-----+-----+-----+
| 101 | Ravi | CSE |
| 102 | Priya | ECE |
| 104 | Meena | CSE |
| 105 | Kiran | ECE |
| 106 | Rahul | IT |
| 107 | Sneha | CSE |
+-----+-----+-----+
6 rows in set (0.00 sec)

mysql> DELETE FROM HeapFile
-> WHERE Record_ID = 103;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> SELECT * FROM HeapFile;
+-----+-----+-----+
| Record_ID | Student_Name | Department |
+-----+-----+-----+
| 101 | Ravi | CSE |
| 102 | Priya | ECE |
| 104 | Meena | CSE |
| 105 | Kiran | ECE |
| 106 | Rahul | IT |
| 107 | Sneha | CSE |
+-----+-----+-----+
6 rows in set (0.00 sec)

mysql> INSERT INTO SequentialFile VALUES
-> (108, 'Kavya', 'CSE');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO HeapFile VALUES
-> (108, 'Kavya', 'CSE');
Query OK, 1 row affected (0.02 sec)

```

```
mysql> INSERT INTO SequentialFile VALUES
-> (108,'Kavya','CSE');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO HeapFile VALUES
-> (108,'Kavya','CSE');
Query OK, 1 row affected (0.02 sec)

mysql>
mysql> SELECT * FROM SequentialFile ORDER BY Record_ID;
+-----+-----+-----+
| Record_ID | Student_Name | Department |
+-----+-----+-----+
| 101 | Ravi | CSE |
| 102 | Priya | ECE |
| 104 | Meena | CSE |
| 105 | Kiran | ECE |
| 106 | Rahul | IT |
| 107 | Sneha | CSE |
| 108 | Kavya | CSE |
+-----+-----+-----+
7 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM HeapFile;
+-----+-----+-----+
| Record_ID | Student_Name | Department |
+-----+-----+-----+
| 101 | Ravi | CSE |
| 102 | Priya | ECE |
| 104 | Meena | CSE |
| 105 | Kiran | ECE |
| 106 | Rahul | IT |
| 107 | Sneha | CSE |
| 108 | Kavya | CSE |
+-----+-----+-----+
7 rows in set (0.00 sec)

mysql>
mysql> SELECT
-> 'Sequential File' AS File_Organization,
-> 'Ordered insertion' AS Insertion,
-> 'Requires maintaining order' AS Deletion,
-> 'Efficient ordered retrieval' AS Retrieval
-> UNION ALL
-> SELECT
-> 'Heap File',
-> 'Simple insertion',
-> 'Simple deletion',
-> 'Sequential scan for retrieval';
+-----+-----+-----+-----+
| File_Organization | Insertion | Deletion | Retrieval |
+-----+-----+-----+-----+
| Sequential File | Ordered insertion | Requires maintaining order | Efficient ordered retrieval |
| Heap File | Simple insertion | Simple deletion | Sequential scan for retrieval |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

2Q) Differentiate between clustered indexing and non-clustered indexing with suitable database examples.

ANS

1.Clustered Indexing

A clustered index determines the physical/logical order in which table records are stored based on the indexed column.

Because the data records themselves are organized according to the clustered index, a table can generally have only one clustered index.

For example, consider a student table:

Student_ID	Name	Department
101	Arun	IT
102	Ravi	CSE
103	Priya	ECE
104	Kiran	IT

If Student_ID is the clustered key, the records are organized according to Student_ID.

Advantages:

- Very efficient for searching using the clustered key.
- Excellent for range queries.
- Faster sequential retrieval.
- Reduces additional data lookup for the indexed column.

Disadvantage:

- Insertions and updates can be more expensive if they require maintaining the physical ordering.

Non-Clustered Indexing

A non-clustered index is a separate index structure that stores the indexed values along with a reference to the corresponding table record.

The physical order of the table does not have to match the order of the index.

A table can have multiple non-clustered indexes.

For example:

Student Table

```
|
+---- Index on Student_Name
|
+---- Index on Department
|
+---- Index on Email
```

The indexes help MySQL locate records faster without changing the basic organization of the table.

Advantages:

- Faster searching on frequently queried columns.
- Multiple non-clustered indexes can be created.
- Useful for different search conditions.

Disadvantages:

- Requires additional storage.
- Insert, update, and delete operations may become slower because indexes must also be maintained.

2. Difference Between Clustered and Non-Clustered Indexing

Feature	Clustered Index	Non-Clustered Index
Data arrangement	Data is organized according to the index	Data remains separately organized
Number per table	Generally one	Multiple possible
Storage	Index and data are closely associated	Separate index structure
Searching	Very efficient	Efficient
Range queries	Very efficient	Efficient
Additional storage	Comparatively less	Requires additional storage
Insert/update	May be slower when order changes	Index must be updated
Example	Primary key in InnoDB	Secondary index on Name/Department
Best use	Ordering and range retrieval	Searching different columns

```
mysql> INSERT INTO ClusteredExample VALUES
-> (101,'Ravi','CSE',85),
-> (102,'Priya','CSE',70),
-> (103,'Arun','IT',91),
-> (104,'Meena','CSE',88),
-> (105,'Mina','CSE',90),
-> (106,'Rahul','IT',95),
-> (107,'Sneha','CSE',82);
Query OK, 7 rows affected (0.02 sec)
Records: 7 Duplicates: 0 Warnings: 0

mysql> INSERT INTO NonClusteredExample VALUES
-> (101,'Ravi','CSE',85),
-> (102,'Priya','CSE',70),
-> (103,'Arun','IT',91),
-> (104,'Meena','CSE',88),
-> (105,'Mina','CSE',90),
-> (106,'Rahul','IT',95),
-> (107,'Sneha','CSE',82);
Query OK, 7 rows affected (0.01 sec)
Records: 7 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_clustered_department
-> ON ClusteredExample(Department);
Query OK, 0 rows affected (0.00 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_nonclustered_marks
-> ON NonClusteredExample(Marks);
Query OK, 0 rows affected (0.10 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SHOW INDEX FROM ClusteredExample;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ClusteredExample | 0 | PRIMARY | 1 | Student_ID | A | 7 | NULL | NULL | NULL | YES | BTREE | | | YES | NULL |
| ClusteredExample | 1 | idx_clustered_department | 1 | Department | A | 3 | NULL | NULL | YES | BTREE | | | | YES | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.03 sec)

mysql> SHOW INDEX FROM NonClusteredExample;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| NonClusteredExample | 0 | PRIMARY | 1 | Student_ID | A | 7 | NULL | NULL | NULL | YES | BTREE | | | YES | NULL |
| NonClusteredExample | 1 | idx_nonclustered_marks | 1 | Marks | A | 3 | NULL | NULL | YES | BTREE | | | | YES | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)

mysql> SELECT *
-> FROM ClusteredExample
-> WHERE Department = 'CSE';
+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | Marks |
+-----+-----+-----+-----+
| 101 | Ravi | CSE | 85 |
| 104 | Meena | CSE | 88 |
| 107 | Sneha | CSE | 82 |
+-----+-----+-----+-----+
```

```

mysql> SELECT *
-> FROM NonClusteredExample
-> WHERE Marks >= 85
-> ORDER BY Marks;
+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | Marks |
+-----+-----+-----+-----+
| 101 | Ravi | CSE | 85 |
| 104 | Meena | CSE | 85 |
| 103 | Arun | IT | 91 |
| 106 | Rahul | IT | 95 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> EXPLAIN SELECT *
-> FROM ClusteredExample
-> WHERE Department = 'CSE';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | ClusteredExample | NULL | ref | idx_clustered_department | idx_clustered_department | 203 | const | 3 | 100.00 | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.01 sec)

mysql> EXPLAIN SELECT *
-> FROM NonClusteredExample
-> WHERE Marks >= 85
-> ORDER BY Marks;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | NonClusteredExample | NULL | range | idx_nonclustered_marks | idx_nonclustered_marks | 5 | NULL | 4 | 100.00 | Using index condition |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> SELECT
-> 'Clustered Index' AS Index_Type,
-> 'Organizes data according to index order' AS Data_Organization,
-> 'Generally efficient for ordered/range access' AS Access_Type
-> UNION ALL
-> SELECT
-> 'Non-Clustered Index',
-> 'Separate index structure points to data records',
-> 'Efficient for indexed search conditions';
+-----+-----+-----+
| Index_Type | Data_Organization | Access_Type |
+-----+-----+-----+
| Clustered Index | Organizes data according to index order | Generally efficient for ordered/range access |
| Non-Clustered Index | Separate index structure points to data records | Efficient for indexed search conditions |
+-----+-----+-----+
2 rows in set (0.00 sec)

```

3Q) Compare primary indexing and secondary indexing based on storage requirements and search performance.

ANS

1. Theory

Indexing is a database technique used to speed up the retrieval of records. Instead of searching every record in a table, the database uses an index to locate the required records more quickly.

Primary Indexing

A primary index is created on the primary key of a table. The records are generally maintained in the same order as the primary key.

For example:

Student_ID	Name	Department
101	Arun	IT
102	Ravi	CSE
103	Priya	ECE
104	Kiran	IT

Here, Student_ID can be used as the primary index.

Characteristics:

- Usually created on the primary key.
- Search using the primary key is very efficient.
- The indexed key is unique.

- In an InnoDB table, the primary key is the clustered index.
- Requires index storage in addition to the table data.
- Particularly effective for exact searches and range searches on the primary key.

Secondary Indexing

A secondary index is created on a column other than the primary key.

For example:

Student_ID → Primary Index

Department → Secondary Index

Student_Name → Secondary Index

A secondary index contains indexed values and references to the corresponding table records.

Example:

Department	Student_ID
CSE	102
ECE	103
IT	101
IT	104

Here, Department can be used as a secondary index.

Characteristics:

- Created on non-primary-key columns.
- Multiple secondary indexes can exist on a table.
- Useful when queries frequently search non-primary-key attributes.
- Requires additional storage.
- Search can be fast, but accessing the actual row may require an additional lookup.

2. Comparison

Feature	Primary Indexing	Secondary Indexing
Index column	Primary key	Non-primary-key column
Uniqueness	Usually unique	May contain duplicate values
Number per table	One primary key/index	Multiple possible
Storage requirement	Requires index storage; InnoDB primary index contains table records	Requires additional index storage
Search performance	Very fast for primary-key searches	Fast for indexed secondary-column searches
Range search	Very efficient	Efficient, depending on query/selectivity
Duplicate values	Not allowed for primary key	Allowed
Example	Student_ID	Department, Student_Name
MySQL InnoDB	Clustered primary-key index	Secondary index

Storage Requirement

A primary index in InnoDB is special because the primary-key B-tree is the clustered index, whose leaf pages contain the complete row data.

A secondary index stores the secondary indexed column values plus the primary-key value used to locate the complete row.

Therefore, creating many secondary indexes can consume significant additional storage.

Search Performance

For:

WHERE Student_ID = 103

the primary index can directly locate the row.

For:

WHERE Department = 'CSE'

```
mysql> CREATE TABLE SecondaryIndexExample (
  -> Student_ID INT NOT NULL KEY,
  -> Student_Name VARCHAR(100),
  -> Department VARCHAR(50),
  -> Marks INT
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO PrimaryIndexExample VALUES
  -> (101,'Ravi','CSE',85),
  -> (102,'Priya','ECE',78),
  -> (103,'Arun','IT',91),
  -> (104,'Meena','CSE',88),
  -> (105,'Miran','ECE',74),
  -> (106,'Rahul','IT',95),
  -> (107,'Sneha','CSE',82),
  -> (108,'Mavya','ECE',90);
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO SecondaryIndexExample VALUES
  -> (101,'Ravi','CSE',85),
  -> (102,'Priya','ECE',78),
  -> (103,'Arun','IT',91),
  -> (104,'Meena','CSE',88),
  -> (105,'Miran','ECE',74),
  -> (106,'Rahul','IT',95),
  -> (107,'Sneha','CSE',82),
  -> (108,'Mavya','ECE',90);
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_secondary_department
  -> ON SecondaryIndexExample(Department);
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SHOW INDEX FROM PrimaryIndexExample;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| primaryindexexample | 0 | PRIMARY | 1 | Student_ID | A | 8 | NULL | NULL | NULL | BTREE | | | YES | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.03 sec)

mysql> SHOW INDEX FROM SecondaryIndexExample;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| secondaryindexexample | 0 | PRIMARY | 1 | Student_ID | A | 8 | NULL | NULL | NULL | BTREE | | | YES | NULL |
| secondaryindexexample | 1 | idx_secondary_department | 1 | Department | A | 3 | NULL | NULL | YES | BTREE | | | YES | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)

mysql> SELECT *
  -> FROM PrimaryIndexExample
  -> WHERE Student_ID = 106;
+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | Marks |
+-----+-----+-----+-----+
| 106 | Rahul | IT | 95 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> SELECT *
  -> FROM SecondaryIndexExample
  -> WHERE Department = 'CSE';
+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | Marks |
+-----+-----+-----+-----+
| 101 | Ravi | CSE | 85 |
| 104 | Meena | CSE | 88 |
| 107 | Sneha | CSE | 82 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> EXPLAIN SELECT *
  -> FROM PrimaryIndexExample
  -> WHERE Student_ID = 106;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | PrimaryIndexExample | NULL | const | PRIMARY | PRIMARY | 4 | const | 1 | 100.00 | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> EXPLAIN SELECT *
  -> FROM SecondaryIndexExample
  -> WHERE Department = 'CSE';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | SecondaryIndexExample | NULL | ref | idx_secondary_department | idx_secondary_department | 203 | const | 3 | 100.00 | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> SELECT
  -> 'Primary Index' AS Index_Type,
  -> 'Usually one index associated with the primary key' AS Storage_Requirement,
  -> 'Fast lookup using primary key' AS Search_Performance
  -> UNION ALL
  -> SELECT
  -> 'Secondary Index',
  -> 'Additional storage required for the indexed attribute',
  -> 'Fast search on non-primary indexed attributes';
+-----+-----+-----+
| Index_Type | Storage_Requirement | Search_Performance |
+-----+-----+-----+
| Primary Index | Usually one index associated with the primary key | Fast lookup using primary key |
| Secondary Index | Additional storage required for the indexed attribute | Fast search on non-primary indexed attributes |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT *
  -> FROM SecondaryIndexExample
  -> WHERE Department IN ('CSE','ECE')
  -> ORDER BY Department, Student_ID;
+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | Marks |
+-----+-----+-----+-----+
| 101 | Ravi | CSE | 85 |
| 104 | Meena | CSE | 88 |
| 107 | Sneha | CSE | 82 |
| 102 | Priya | ECE | 78 |
| 105 | Kiran | ECE | 74 |
| 108 | Mavya | ECE | 90 |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql> |
```

4Q) Analyze the differences between static hashing and dynamic hashing in database systems.

ANS

1. Theory

Hashing is a file organization and indexing technique used to quickly locate records using a hash function. The hash function converts a search key into a bucket or storage location.

For example:

Student_ID → Hash Function → Bucket Number

101 → h(key) → Bucket 1

102 → h(key) → Bucket 2

103 → h(key) → Bucket 3

The main types are static hashing and dynamic hashing.

2. Static Hashing

In static hashing, the number of buckets is fixed when the hash file is created.

For example, suppose there are 4 buckets and the hash function is:

$h(\text{Student_ID}) = \text{Student_ID} \text{ MOD } 4$

Then:

Student ID	Hash Calculation	Bucket
101	$101 \text{ MOD } 4 = 1$	1
102	$102 \text{ MOD } 4 = 2$	2
103	$103 \text{ MOD } 4 = 3$	3
104	$104 \text{ MOD } 4 = 0$	0

Advantages

- Simple to implement.
- Fast exact-match searches.
- Easy to understand and maintain.
- Good when the number of records is relatively stable.

Disadvantages

- Number of buckets cannot easily grow with the data.
- More records can cause **collisions**.
- Overflow buckets may be required.
- Performance can decrease when the file becomes large.

3. Dynamic Hashing

In dynamic hashing, the number of buckets can grow or shrink as the amount of data changes.

When the number of records increases, new buckets can be created and records can be redistributed.

Two common dynamic hashing techniques are:

1. **Extendible hashing**
2. **Linear hashing**

Conceptually:

Initial:

Bucket 0

Bucket 1

More records inserted:

Bucket 0

Bucket 1

Bucket 2

Bucket 3

The system dynamically adjusts the number of buckets to reduce overflow.

Advantages

- Adapts to changes in the number of records.
- Reduces overflow problems.
- Maintains good search performance as the file grows.
- Suitable for databases with frequently changing data sizes.

Disadvantages

- More complex than static hashing.
- Bucket splitting requires additional processing.
- Requires additional storage and management overhead

```

mysql>
mysql> CREATE TABLE StaticHash (
  -> Student_ID INT PRIMARY KEY,
  -> Student_Name VARCHAR(100),
  -> Department VARCHAR(50),
  -> Bucket_No INT
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE DynamicHash (
  -> Student_ID INT PRIMARY KEY,
  -> Student_Name VARCHAR(100),
  -> Department VARCHAR(50),
  -> Bucket_No INT
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO StaticHash VALUES
  -> (101,'Ravi','CSE',MOD(101,4)),
  -> (102,'Priya','ECE',MOD(102,4)),
  -> (103,'Arun','IT',MOD(103,4)),
  -> (104,'Meena','CSE',MOD(104,4)),
  -> (105,'Kiran','ECE',MOD(105,4)),
  -> (106,'Rahul','IT',MOD(106,4)),
  -> (107,'Sneha','CSE',MOD(107,4)),
  -> (108,'Kavya','ECE',MOD(108,4));
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> SELECT
  -> Bucket_No,
  -> GROUP_CONCAT(Student_ID ORDER BY Student_ID SEPARATOR ' -> ') AS Student_Chain
  -> FROM StaticHash
  -> GROUP BY Bucket_No
  -> ORDER BY Bucket_No;
+-----+-----+
| Bucket_No | Student_Chain |
+-----+-----+
| 0 | 104 -> 108 |
| 1 | 101 -> 105 |
| 2 | 102 -> 106 |
| 3 | 103 -> 107 |
+-----+-----+
4 rows in set (0.00 sec)

mysql> INSERT INTO DynamicHash VALUES
  -> (101,'Ravi','CSE',MOD(101,2)),
  -> (102,'Priya','ECE',MOD(102,2)),
  -> (103,'Arun','IT',MOD(103,2)),
  -> (104,'Meena','CSE',MOD(104,2)),
  -> (105,'Kiran','ECE',MOD(105,2)),
  -> (106,'Rahul','IT',MOD(106,2)),
  -> (107,'Sneha','CSE',MOD(107,2)),
  -> (108,'Kavya','ECE',MOD(108,2));
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> SELECT
  -> Bucket_No,
  -> GROUP_CONCAT(Student_ID ORDER BY Student_ID SEPARATOR ' -> ') AS Student_Chain
  -> FROM DynamicHash
  -> GROUP BY Bucket_No
  -> ORDER BY Bucket_No;
+-----+-----+
| Bucket_No | Student_Chain |
+-----+-----+
| 0 | 102 -> 104 -> 106 -> 108 |
| 1 | 101 -> 103 -> 105 -> 107 |
+-----+-----+
2 rows in set (0.00 sec)

mysql> UPDATE DynamicHash
  -> SET Bucket_No = MOD(Student_ID,4);
Query OK, 4 rows affected (0.01 sec)
Rows matched: 8 Changed: 4 Warnings: 0

```

```
mysql> SELECT
  -> Bucket_No,
  -> GROUP_CONCAT(Student_ID ORDER BY Student_ID SEPARATOR ' -> ') AS Student_Chain
  -> FROM DynamicHash
  -> GROUP BY Bucket_No
  -> ORDER BY Bucket_No;
```

Bucket_No	Student_Chain
0	104 -> 108
1	101 -> 105
2	102 -> 106
3	103 -> 107

4 rows in set (0.00 sec)

```
mysql> SELECT *
  -> FROM StaticHash
  -> WHERE Bucket_No = MOD(106,4)
  -> AND Student_ID = 106;
```

Student_ID	Student_Name	Department	Bucket_No
106	Rahul	IT	2

1 row in set (0.00 sec)

```
mysql> SELECT *
  -> FROM DynamicHash
  -> WHERE Bucket_No = MOD(106,4)
  -> AND Student_ID = 106;
```

Student_ID	Student_Name	Department	Bucket_No
106	Rahul	IT	2

1 row in set (0.00 sec)

```
mysql> SELECT
  -> 'Static Hashing' AS Hashing_Type,
  -> 'Fixed number of buckets' AS Bucket_Management,
  -> 'OverFlow may occur when buckets become full' AS Scalability,
  -> 'Simple and fast for fixed-size data' AS Performance
  -> UNION ALL
  -> SELECT
  -> 'Dynamic Hashing',
  -> 'Buckets can expand or reorganize',
  -> 'Handles growth more effectively',
  -> 'Better for changing data size';
```

Hashing_Type	Bucket_Management	Scalability	Performance
Static Hashing	Fixed number of buckets	OverFlow may occur when buckets become full	Simple and fast for fixed-size data
Dynamic Hashing	Buckets can expand or reorganize	Handles growth more effectively	Better for changing data size

2 rows in set (0.00 sec)

```
mysql> SELECT
  -> COUNT(*) AS Total_Records,
  -> COUNT(DISTINCT Bucket_No) AS Active_Buckets,
  -> MIN(Bucket_No) AS Minimum_Bucket,
  -> MAX(Bucket_No) AS Maximum_Bucket
  -> FROM DynamicHash;
```

Total_Records	Active_Buckets	Minimum_Bucket	Maximum_Bucket
8	4	0	3

1 row in set (0.00 sec)

5Q) Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.

ANS

1. Theory

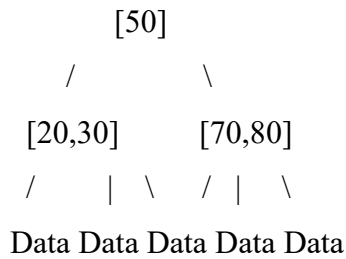
Both B-Tree and B+ Tree are balanced tree-based indexing techniques used in database systems. They keep the tree balanced so that searching, insertion, and deletion can be performed efficiently.

They are especially useful for large-scale databases because their height remains small even when the number of records becomes very large.

2. B-Tree Indexing

A B-Tree (Balanced Tree) stores keys and data/record pointers in both internal nodes and leaf nodes.

Example:



A search can sometimes finish at an internal node because the actual record or pointer may be stored there.

Characteristics

- Balanced tree structure.
- Keys can be stored in internal and leaf nodes.
- Data pointers can exist in internal and leaf nodes.
- Search can terminate at an internal node.
- Supports insertion, deletion, and searching efficiently.
- Suitable for exact-match searches and range searches.

Advantages

- Efficient searching.
- Balanced structure gives predictable performance.
- Good for insert, delete, and search operations.

Disadvantages

- Internal nodes contain data pointers, so fewer keys may fit in each node.
- Range traversal is less convenient compared with B+ Trees.

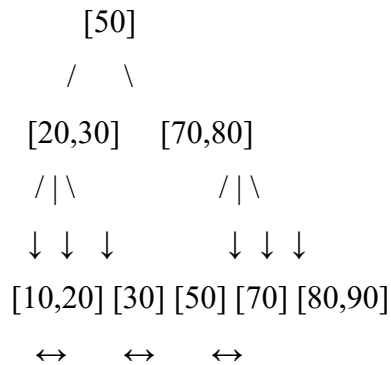
3. B+ Tree Indexing

A B+ Tree is a variation of the B-Tree commonly used in database indexing.

In a B+ Tree:

- Internal nodes contain only keys and child pointers.
- Leaf nodes contain the actual record pointers/data.
- Leaf nodes are linked together for efficient sequential and range access.

Example:



The arrows between leaf nodes allow the database to scan records sequentially.

Characteristics

- All actual data/record pointers are stored at the leaf level.
- Internal nodes act as routing/index nodes.
- Leaf nodes are linked.
- Excellent for range queries.
- High fan-out means fewer tree levels.
- Widely used for database indexes.

4. B-Tree vs B+ Tree

Feature	B-Tree	B+ Tree
Data storage	Internal and leaf nodes	Leaf nodes only
Internal nodes	Keys + data pointers	Keys + child pointers
Leaf nodes	Data + keys	Data/record pointers + keys
Leaf nodes linked	Usually no	Yes
Search	Can end at internal node	Usually reaches leaf
Range queries	Good	Excellent
Sequential access	Less efficient	Very efficient
Fan-out	Lower	Higher
Tree height	May be slightly higher	Usually smaller
Large databases	Good	Better suited
Database usage	Less common	Very common

```

-> );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO BTreeIndex VALUES
-> (101,'Ravi',85),
-> (104,'Meena',88),
-> (107,'Sneha',82),
-> (110,'Arun',91),
-> (115,'Priya',78),
-> (120,'Kiran',90),
-> (125,'Rahul',95),
-> (130,'Kavya',87),
-> (135,'Anu',84),
-> (140,'Vijay',92);
Query OK, 10 rows affected (0.03 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> INSERT INTO BPlusTreeIndex VALUES
-> (101,'Ravi',85),
-> (104,'Meena',88),
-> (107,'Sneha',82),
-> (110,'Arun',91),
-> (115,'Priya',78),
-> (120,'Kiran',90),
-> (125,'Rahul',95),
-> (130,'Kavya',87),
-> (135,'Anu',84),
-> (140,'Vijay',92);
Query OK, 10 rows affected (0.01 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_btree_student
-> ON BTreeIndex(Student_ID);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
mysql> CREATE INDEX idx_bplustree_student
-> ON BPlusTreeIndex(Student_ID);
Query OK, 0 rows affected (0.07 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT *
-> FROM BTreeIndex
-> ORDER BY Student_ID;
+-----+-----+-----+
| Student_ID | Student_Name | Marks |
+-----+-----+-----+
| 101 | Ravi | 85 |
| 104 | Meena | 88 |
| 107 | Sneha | 82 |
| 110 | Arun | 91 |
| 115 | Priya | 78 |
| 120 | Kiran | 90 |
| 125 | Rahul | 95 |
| 130 | Kavya | 87 |
| 135 | Anu | 84 |
| 140 | Vijay | 92 |
+-----+-----+-----+
10 rows in set (0.00 sec)

mysql> SELECT *
-> FROM BPlusTreeIndex
-> ORDER BY Student_ID;
+-----+-----+-----+
| Student_ID | Student_Name | Marks |
+-----+-----+-----+
| 101 | Ravi | 85 |
| 104 | Meena | 88 |
| 107 | Sneha | 82 |
| 110 | Arun | 91 |
| 115 | Priya | 78 |
| 120 | Kiran | 90 |
| 125 | Rahul | 95 |
| 130 | Kavya | 87 |
| 135 | Anu | 84 |
| 140 | Vijay | 92 |
+-----+-----+-----+
10 rows in set (0.00 sec)

```

```

+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | BTreeIndex | NULL | const | PRIMARY,idx_btree_student | PRIMARY | 4 | const | 1 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> EXPLAIN SELECT *
-> FROM BPlusTreeIndex
-> WHERE Student_ID = 120;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | BPlusTreeIndex | NULL | const | PRIMARY,idx_bplustree_student | PRIMARY | 4 | const | 1 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> SELECT *
-> FROM BTreeIndex
-> WHERE Student_ID BETWEEN 110 AND 130
-> ORDER BY Student_ID;
+----+-----+-----+
| Student_ID | Student_Name | Marks |
+----+-----+-----+
| 110 | Arun | 91 |
| 115 | Priya | 78 |
| 120 | Kiran | 90 |
| 125 | Rahul | 95 |
| 130 | Kavya | 87 |
+----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT *
-> FROM BPlusTreeIndex
-> WHERE Student_ID BETWEEN 110 AND 130
-> ORDER BY Student_ID;
+----+-----+-----+
| Student_ID | Student_Name | Marks |
+----+-----+-----+
| 110 | Arun | 91 |
| 115 | Priya | 78 |
| 120 | Kiran | 90 |
| 125 | Rahul | 95 |
| 130 | Kavya | 87 |
+----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT
-> 'B-Tree' AS Index_Type,
-> 'Keys and records may exist in internal and leaf nodes' AS Data_Organization,
-> 'Good for exact-match and range searches' AS Search_Performance
-> UNION ALL
-> SELECT
-> 'B+ Tree',
-> 'Records are stored at leaf nodes and leaf nodes are linked',
-> 'Very efficient for range and sequential searches';
+----+-----+-----+
| Index_Type | Data_Organization | Search_Performance |
+----+-----+-----+
| B-Tree | Keys and records may exist in internal and leaf nodes | Good for exact-match and range searches |
| B+ Tree | Records are stored at leaf nodes and leaf nodes are linked | Very efficient for range and sequential searches |
+----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT
-> 'B-Tree' AS Index_Type,
-> 'Suitable for exact and range queries' AS Advantage,
-> 'Range traversal is less efficient than B+ Tree' AS Limitation
-> UNION ALL
-> SELECT
-> 'B+ Tree',
-> 'Excellent sequential and range access',
-> 'Requires additional leaf-node links';
+----+-----+-----+
| Index_Type | Advantage | Limitation |
+----+-----+-----+
| B-Tree | Suitable for exact and range queries | Range traversal is less efficient than B+ Tree |
| B+ Tree | Excellent sequential and range access | Requires additional leaf-node links |
+----+-----+-----+
2 rows in set (0.00 sec)

```

6Q) Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.

ANS

1. Theory

Dynamic hashing allows the number of buckets in a hash file to change as the amount of data grows. Two important dynamic hashing techniques are Linear Hashing and Extendible Hashing.

2. Linear Hashing

Linear hashing expands the hash file gradually, one bucket at a time.

It does not require a large directory. When a bucket becomes full or the load factor becomes high, a bucket is split according to a predefined sequence.

Example

Initially:

Bucket 0

Bucket 1

Bucket 2

Bucket 3

When more records are inserted:

Bucket 0 → split

Bucket 1

Bucket 2

Bucket 3

Bucket 4 ← new bucket

Later:

Bucket 0

Bucket 1 → split

Bucket 2

Bucket 3

Bucket 4

Bucket 5 ← new bucket

Characteristics

- Buckets are split incrementally.
- No large directory is required.
- Expansion happens gradually.
- Suitable for databases where data grows continuously.
- Overflow can be handled using overflow pages/buckets.

Advantages

- Simple bucket management.
- Smooth expansion.
- No large directory overhead.
- Good scalability for continuously growing data.

Disadvantages

- Bucket splitting can temporarily add overhead.
- Hash calculations are more complex because multiple hash levels can exist.

- Lookup logic is somewhat more complicated during expansion.

3. Extendible Hashing

Extendible hashing uses a directory containing pointers to buckets.

The directory is controlled by a global depth, while each bucket has a local depth.

Example:

Directory

00 —————→ Bucket A

01 —————→ Bucket B

10 —————→ Bucket C

11 —————→ Bucket D

When a bucket becomes full, it is split.

If the current directory cannot accommodate the split, the directory doubles in size.

Example

Before splitting:

00 → Bucket A

01 → Bucket B

10 → Bucket C

11 → Bucket D

After directory expansion:

000 → Bucket A

001 → Bucket B

010 → Bucket C

011 → Bucket D

100 → Bucket A

101 → Bucket B

110 → Bucket C

111 → Bucket D

Not every directory entry necessarily points to a different bucket.

Characteristics

- Uses a directory.
- Directory can double when required.
- Buckets split independently.
- Uses global and local depth.

- Provides very efficient exact-match searching.

Advantages

- Fast searching.
- Efficient handling of bucket overflow.
- Dynamic growth.
- Avoids long overflow chains.

Disadvantages

- Directory requires additional memory/storage.
- Directory doubling can cause overhead.
- More complex than linear hashing.

4. Linear Hashing vs Extendible Hashing

Feature	Linear Hashing	Extendible Hashing
Bucket management	Buckets split one at a time	Buckets split when they overflow
Directory	No directory required	Uses a directory
Growth	Gradual	Directory may double
Bucket splitting	Predetermined linear order	Based on overflowing bucket
Memory overhead	Lower	Higher due to directory
Scalability	Very good for continuous growth	Very good for rapidly changing data
Overflow handling	May use overflow buckets	Bucket splitting reduces overflow
Search	Efficient	Very efficient
Complexity	Relatively simpler	More complex
Best suited for	Continuously growing files	Dynamic/random growth
Main advantage	Smooth bucket expansion	Fast direct bucket access

5. Main Difference in Bucket Management

Linear Hashing

Bucket 0 → Bucket 1 → Bucket 2 → Bucket 3

↓

Split

↓

Bucket 4 added

One bucket is split at a time.

Extendible Hashing

Directory

↓

Bucket overflow

↓

Bucket split

↓

Directory may double

A directory controls bucket access.

6. Scalability

Linear Hashing

Linear hashing is highly scalable because it grows gradually.

4 buckets

↓

5 buckets

↓

6 buckets

↓

7 buckets

↓

8 buckets

Only the required bucket is split at a time.

```

mysql>
mysql> CREATE TABLE ExtendibleHashing (
    ->     Student_ID INT,
    ->     Student_Name VARCHAR(100),
    ->     Bucket_No INT
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO LinearHashing VALUES
    -> (101,'Ravi',1),
    -> (102,'Priya',2),
    -> (103,'Arun',3),
    -> (104,'Meena',0),
    -> (105,'Kiran',1),
    -> (106,'Rahul',2),
    -> (107,'Sneha',3),
    -> (108,'Kavya',0);
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO ExtendibleHashing VALUES
    -> (101,'Ravi',1),
    -> (102,'Priya',2),
    -> (103,'Arun',3),
    -> (104,'Meena',0),
    -> (105,'Kiran',1),
    -> (106,'Rahul',2),
    -> (107,'Sneha',3),
    -> (108,'Kavya',0);
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> SELECT
    -> Bucket_No,
    -> GROUP_CONCAT(Student_ID ORDER BY Student_ID SEPARATOR ' -> ') AS Student_Chain
    -> FROM LinearHashing
    -> GROUP BY Bucket_No
    -> ORDER BY Bucket_No;
+-----+-----+
| Bucket_No | Student_Chain |
+-----+-----+
| 0 | 104 -> 108 |
| 1 | 101 -> 105 |
| 2 | 102 -> 106 |
| 3 | 103 -> 107 |
+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT
    -> Bucket_No,
    -> GROUP_CONCAT(Student_ID ORDER BY Student_ID SEPARATOR ' -> ') AS Student_Chain
    -> FROM ExtendibleHashing
    -> GROUP BY Bucket_No
    -> ORDER BY Bucket_No;
+-----+-----+
| Bucket_No | Student_Chain |
+-----+-----+
| 0 | 104 -> 108 |
| 1 | 101 -> 105 |
| 2 | 102 -> 106 |
| 3 | 103 -> 107 |
+-----+-----+
4 rows in set (0.00 sec)

```

```

+-----+-----+-----+
| Student_ID | Student_Name | Bucket_No |
+-----+-----+-----+
| 106 | Rahul | 2 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT *
-> FROM ExtendibleHashing
-> WHERE Bucket_No = MOD(106,4)
-> AND Student_ID = 106;
+-----+-----+-----+
| Student_ID | Student_Name | Bucket_No |
+-----+-----+-----+
| 106 | Rahul | 2 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT
-> 'Linear Hashing' AS Hashing_Type,
-> 'Buckets grow gradually as records increase' AS Bucket_Management,
-> 'Good scalability with controlled bucket growth' AS Scalability
-> UNION ALL
-> SELECT
-> 'Extendible Hashing',
-> 'Directory expands and buckets split when required',
-> 'Highly scalable for dynamically growing data';
+-----+-----+-----+
| Hashing_Type | Bucket_Management | Scalability |
+-----+-----+-----+
| Linear Hashing | Buckets grow gradually as records increase | Good scalability with controlled bucket growth |
| Extendible Hashing | Directory expands and buckets split when required | Highly scalable for dynamically growing data |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT
-> 'Linear Hashing' AS Hashing_Type,
-> 'Simple bucket expansion' AS Advantage,
-> 'May require overflow handling' AS Limitation
-> UNION ALL
-> SELECT
-> 'Extendible Hashing',
-> 'Reduces overflow using bucket splitting',
-> 'Directory requires additional memory';
+-----+-----+-----+
| Hashing_Type | Advantage | Limitation |
+-----+-----+-----+
| Linear Hashing | Simple bucket expansion | May require overflow handling |
| Extendible Hashing | Reduces overflow using bucket splitting | Directory requires additional memory |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT
-> COUNT(*) AS Total_Records,
-> COUNT(DISTINCT Bucket_No) AS Active_Buckets,
-> MIN(Bucket_No) AS Minimum_Bucket,
-> MAX(Bucket_No) AS Maximum_Bucket
-> FROM LinearHashing;
+-----+-----+-----+
| Total_Records | Active_Buckets | Minimum_Bucket | Maximum_Bucket |
+-----+-----+-----+
| 8 | 4 | 0 | 3 |
+-----+-----+-----+
1 row in set (0.00 sec)

```

7Q) Compare dense indexing and sparse indexing with respect to memory usage and access speed.

ANS

1. Theory

An index is an additional data structure used to locate records quickly without scanning the entire table.

The two common types are:

- Dense Index
- Sparse Index

The main difference is how many index entries are stored.

2. Dense Indexing

In dense indexing, the index contains an entry for every search-key value or every record in the data file.

Example

Suppose the data file contains:

Student_ID	Name	Department
101	Arun	IT
102	Ravi	CSE
103	Priya	ECE
104	Kiran	IT
105	Meena	CSE

A dense index can contain:

Index Key	Record Pointer
101	→ Record 101
102	→ Record 102
103	→ Record 103
104	→ Record 104
105	→ Record 105

Therefore, every record has an index entry.

Advantages

- Very fast searching.
- Direct access to records.
- Suitable for frequently searched data.
- Useful when the data file is not ordered.

Disadvantages

- Requires more memory/storage.
- Insertions and deletions require more index maintenance.
- Large datasets can result in a very large index.

3. Sparse Indexing

In **sparse indexing**, the index contains entries for **only some search-key values**, usually one entry per data block.

For example:

Index Key	Block
101	→ Block 1
104	→ Block 2
107	→ Block 3

If the records are sorted:

Index



101 → Block 1 → 101,102,103

104 → Block 2 → 104,105,106

107 → Block 3 → 107,108,109

To find 105, the database first finds the index entry 104, then searches within Block 2.

Advantages

- Requires less memory.
- Smaller index size.
- Less index maintenance.
- Efficient for large, ordered files.

Disadvantages

- Search can be slightly slower than dense indexing.
- The data file generally needs to be ordered according to the search key.
- May require an additional search within the selected block.

4. Dense vs Sparse Indexing

Feature	Dense Indexing	Sparse Indexing
Index entries	Every record/key	Selected records/blocks
Memory usage	High	Low
Index size	Large	Smaller
Search speed	Faster	Slightly slower
Data ordering	Not necessarily required	Usually required
Insert/Delete maintenance	Higher	Lower
Storage overhead	High	Low
Suitable for	Frequently searched data	Large ordered files
Access	Direct/near-direct	Index + block search

5. Memory Usage

Suppose there are **1,000,000 records**.

If a dense index has one entry per record:

1,000,000 records



1,000,000 index entries

If a sparse index has one entry per 100 records:

1,000,000 records



10,000 index entries

Therefore:

Dense indexing consumes more memory, while sparse indexing saves memory.

6. Access Speed

Dense Index

Searching for Student_ID = 105:

Index



105



Record 105

The record can be located very quickly.

Sparse Index

Index



101 → Block 1

104 → Block 2



Search inside Block 2



105

Therefore:

Dense indexing generally provides faster access, while sparse indexing uses less memory.

```
mysql>
mysql> CREATE TABLE SparseIndex (
  ->     Student_ID INT,
  ->     Student_Name VARCHAR(100),
  ->     Marks INT
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO DenseIndex VALUES
  -> (101,'Ravi',85),
  -> (105,'Kiran',74),
  -> (110,'Arun',91),
  -> (115,'Priya',78),
  -> (120,'Meena',88),
  -> (125,'Rahul',95),
  -> (130,'Sneha',82);
Query OK, 7 rows affected (0.01 sec)
Records: 7 Duplicates: 0 Warnings: 0

mysql> INSERT INTO SparseIndex VALUES
  -> (101,'Ravi',85),
  -> (105,'Kiran',74),
  -> (110,'Arun',91),
  -> (115,'Priya',78),
  -> (120,'Meena',88),
  -> (125,'Rahul',95),
  -> (130,'Sneha',82);
Query OK, 7 rows affected (0.03 sec)
Records: 7 Duplicates: 0 Warnings: 0

mysql> SELECT *
  -> FROM DenseIndex
  -> ORDER BY Student_ID;
+-----+-----+-----+
| Student_ID | Student_Name | Marks |
+-----+-----+-----+
| 101 | Ravi | 85 |
| 105 | Kiran | 74 |
| 110 | Arun | 91 |
| 115 | Priya | 78 |
| 120 | Meena | 88 |
| 125 | Rahul | 95 |
| 130 | Sneha | 82 |
+-----+-----+-----+
7 rows in set (0.00 sec)

mysql> SELECT *
  -> FROM SparseIndex
  -> ORDER BY Student_ID;
+-----+-----+-----+
| Student_ID | Student_Name | Marks |
+-----+-----+-----+
| 101 | Ravi | 85 |
| 105 | Kiran | 74 |
| 110 | Arun | 91 |
| 115 | Priya | 78 |
| 120 | Meena | 88 |
| 125 | Rahul | 95 |
| 130 | Sneha | 82 |
+-----+-----+-----+
7 rows in set (0.00 sec)
```

```
mysql> SELECT
-> Student_ID AS Index_Key,
-> Student_Name,
-> Marks,
-> 'Dense Index - entry for every record' AS Index_Type
-> FROM DenseIndex
-> ORDER BY Student_ID;
```

Index_Key	Student_Name	Marks	Index_Type
101	Ravi	85	Dense Index - entry for every record
105	Kiran	74	Dense Index - entry for every record
110	Arun	91	Dense Index - entry for every record
115	Priya	78	Dense Index - entry for every record
120	Meena	88	Dense Index - entry for every record
125	Rahul	95	Dense Index - entry for every record
130	Sneha	82	Dense Index - entry for every record

7 rows in set (0.00 sec)

```
mysql> SELECT
-> Student_ID AS Index_Key,
-> Student_Name,
-> Marks,
-> 'Sparse Index - selected entries for blocks' AS Index_Type
-> FROM SparseIndex
-> WHERE Student_ID IN (101,115,130)
-> ORDER BY Student_ID;
```

Index_Key	Student_Name	Marks	Index_Type
101	Ravi	85	Sparse Index - selected entries for blocks
115	Priya	78	Sparse Index - selected entries for blocks
130	Sneha	82	Sparse Index - selected entries for blocks

3 rows in set (0.00 sec)

```
mysql> SELECT
-> 'Dense Index' AS Index_Type,
-> 'Higher memory usage' AS Memory_Usage,
-> 'Faster direct access' AS Access_Speed
-> UNION ALL
-> SELECT
-> 'Sparse Index',
-> 'Lower memory usage',
-> 'Slower than dense index but efficient';
```

Index_Type	Memory_Usage	Access_Speed
Dense Index	Higher memory usage	Faster direct access
Sparse Index	Lower memory usage	Slower than dense index but efficient

2 rows in set (0.00 sec)

```
mysql> SELECT
-> 'Dense Index' AS Index_Type,
-> 'Index entry for every record' AS Organization,
-> 'Higher storage requirement' AS Memory,
-> 'Very fast record access' AS Performance
-> UNION ALL
-> SELECT
-> 'Sparse Index',
-> 'Index entries for selected records/blocks',
-> 'Lower storage requirement',
-> 'Efficient access with additional search';
```

Index_Type	Organization	Memory	Performance
Dense Index	Index entry for every record	Higher storage requirement	Very fast record access
Sparse Index	Index entries for selected records/blocks	Lower storage requirement	Efficient access with additional search

2 rows in set (0.00 sec)

8Q) Analyze the advantages and limitations of indexed sequential file organization versus direct file organization

ANS

1. Theory

File organization defines how records are physically stored and accessed in a database. Two important methods are Indexed Sequential File Organization and Direct File Organization.

2. Indexed Sequential File Organization (ISAM)

In Indexed Sequential File Organization, records are stored sequentially according to a key field, and an index is maintained to locate records quickly.

For example, if Student_ID is the key:

Index

↓

101 → Block 1 → 101, 102, 103

104 → Block 2 → 104, 105, 106

107 → Block 3 → 107, 108, 109

The index first identifies the appropriate block, and then the records are searched sequentially within that block.

Advantages

1. **Fast retrieval** – The index reduces the amount of data that must be searched.
2. **Supports sequential access** – Records can be processed in sorted order.
3. **Good for range queries** – Queries such as Student_ID BETWEEN 100 AND 200 are efficient.
4. **Suitable for large databases** – Indexes make searching large sequential files faster.
5. **Efficient reporting** – Sequential processing is useful for generating reports.

Limitations

1. **Extra storage** is required for indexes.
2. **Insertion can be costly** because records need to maintain sequential order.
3. **Deletion may create gaps** or require index maintenance.
4. **Index maintenance overhead** occurs whenever records are inserted, deleted, or updated.
5. Performance may degrade if there are many overflow records.

3. Direct File Organization

In Direct File Organization, also called hashed file organization, a hash function is used to calculate the location of a record directly.

For example:

Hash Function:

$h(\text{Student_ID}) = \text{Student_ID} \text{ MOD } 5$

101 → 101 MOD 5 → Bucket 1

102 → 102 MOD 5 → Bucket 2

103 → 103 MOD 5 → Bucket 3

104 → 104 MOD 5 → Bucket 4

105 → 105 MOD 5 → Bucket 0

Advantages

1. Very fast exact-match retrieval.
2. No need to scan records sequentially.
3. Efficient insertion when the bucket has available space.
4. Suitable for applications requiring frequent direct access.
5. Good for large datasets with equality searches.

Limitations

1. **Collision can occur** when multiple keys map to the same bucket.
2. Requires collision-handling techniques such as overflow buckets or chaining.
3. **Poor for range queries.**
4. Data is not stored sequentially.
5. Hash function design affects performance.

4. Comparison

Feature	Indexed Sequential File Organization	Direct File Organization
Basic technique	Sequential storage + index	Hash function
Data ordering	Sorted/sequential	Not necessarily sorted
Search	Index → block → record	Hash → bucket → record
Exact search	Fast	Very fast
Range search	Very good	Poor
Sequential processing	Excellent	Poor
Insertion	Moderate	Generally fast
Deletion	Index/order maintenance required	Generally simple
Extra storage	Index required	Hash/bucket storage
Collision	Overflow records may occur	Hash collisions
Best suited for	Range queries and reports	Exact-match queries
Example	Student records sorted by ID	Employee lookup using Employee_ID

5. Advantages and Limitations Analysis

Indexed Sequential

Best when:

Need:

- ✓ Exact search
- ✓ Range search
- ✓ Sequential processing
- ✓ Sorted reports

Its major advantage is that it provides both direct indexed access and sequential access.

Its major limitation is the maintenance cost of the index and sequential ordering.

Direct

Best when:

Need:

- ✓ Very fast exact search
- ✓ Frequent insertions
- ✓ Direct record access

Its major advantage is fast direct access using a hash function.

Its major limitation is that it performs poorly for range-based queries because records are not stored in sorted order.

```
mysql> INSERT INTO IndexedSequentialFile VALUES
```

```
-> (101,'Ravi','CSE',85),
-> (104,'Meena','CSE',88),
-> (106,'Rahul','IT',95),
-> (108,'Kavya','CSE',82),
-> (110,'Arun','IT',91),
-> (115,'Priya','CSE',78),
-> (120,'Kiran','ECE',90);
```

```
Query OK, 7 rows affected (0.01 sec)
```

```
Records: 7 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO DirectFile VALUES
```

```
-> (101,'Ravi','CSE',85),
-> (104,'Meena','CSE',88),
-> (106,'Rahul','IT',95),
-> (108,'Kavya','CSE',82),
-> (110,'Arun','IT',91),
-> (115,'Priya','CSE',78),
-> (120,'Kiran','ECE',90);
```

```
Query OK, 7 rows affected (0.01 sec)
```

```
Records: 7 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT *
```

```
-> FROM IndexedSequentialFile
-> ORDER BY Record_ID;
```

Record_ID	Student_Name	Department	Marks
101	Ravi	CSE	85
104	Meena	CSE	88
106	Rahul	IT	95
108	Kavya	CSE	82
110	Arun	IT	91
115	Priya	CSE	78
120	Kiran	ECE	90

```
7 rows in set (0.00 sec)
```

```
mysql> SELECT *
```

```
-> FROM DirectFile
-> ORDER BY Record_ID;
```

Record_ID	Student_Name	Department	Marks
101	Ravi	CSE	85
104	Meena	CSE	88
106	Rahul	IT	95
108	Kavya	CSE	82
110	Arun	IT	91
115	Priya	CSE	78
120	Kiran	ECE	90

```
7 rows in set (0.00 sec)
```

```
mysql> SELECT *
```

```
-> FROM IndexedSequentialFile
-> WHERE Record_ID = 110;
```

Record_ID	Student_Name	Department	Marks
110	Arun	IT	91

```
1 row in set (0.00 sec)
```

```
mysql> SELECT *
-> FROM DirectFile
-> WHERE Record_ID = 110;
+-----+-----+-----+-----+
| Record_ID | Student_Name | Department | Marks |
+-----+-----+-----+-----+
| 110 | Arun | IT | 91 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT
-> 'Indexed Sequential File' AS File_Organization,
-> 'Records maintained in sorted order with index support' AS Organization,
-> 'Supports sequential and direct access' AS Access_Method
-> UNION ALL
-> SELECT
-> 'Direct File',
-> 'Records accessed directly using record address/key',
-> 'Provides direct/random access';
+-----+-----+-----+
| File_Organization | Organization | Access_Method |
+-----+-----+-----+
| Indexed Sequential File | Records maintained in sorted order with index support | Supports sequential and direct access |
| Direct File | Records accessed directly using record address/key | Provides direct/random access |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT
-> 'Indexed Sequential File' AS File_Organization,
-> 'Supports both sequential and direct retrieval' AS Advantage,
-> 'Requires additional index storage and maintenance' AS Limitation
-> UNION ALL
-> SELECT
-> 'Direct File',
-> 'Very fast direct access to known records',
-> 'Poor for sequential/range-based retrieval';
+-----+-----+-----+
| File_Organization | Advantage | Limitation |
+-----+-----+-----+
| Indexed Sequential File | Supports both sequential and direct retrieval | Requires additional index storage and maintenance |
| Direct File | Very fast direct access to known records | Poor for sequential/range-based retrieval |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT
-> 'Indexed Sequential File' AS File_Organization,
-> 'Moderate' AS Storage_Requirement,
-> 'Fast with index' AS Retrieval_Speed,
-> 'Good for sequential and range access' AS Suitability
-> UNION ALL
-> SELECT
-> 'Direct File',
-> 'Low additional storage',
-> 'Very fast for direct access',
-> 'Best for exact record retrieval';
+-----+-----+-----+-----+
| File_Organization | Storage_Requirement | Retrieval_Speed | Suitability |
+-----+-----+-----+-----+
| Indexed Sequential File | Moderate | Fast with index | Good for sequential and range access |
| Direct File | Low additional storage | Very fast for direct access | Best for exact record retrieval |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

9Q) Compare single-level indexing and multi-level indexing for efficient database retrieval operations.

ANS

1. Theory

Indexing is a technique used in database systems to improve the speed of record retrieval. Instead of scanning the complete data file, the database searches an index to locate the required records.

Two common approaches are:

1. Single-Level Indexing
2. Multi-Level Indexing

2. Single-Level Indexing

In single-level indexing, there is one index structure between the search key and the actual data records.

Structure

Search Key



Single-Level Index



Data File



Required Record

Example:

Student_ID	Record Address
101	→ Record 101
102	→ Record 102
103	→ Record 103
104	→ Record 104

To find 103:

Search



Index



103



Record 103

Advantages

- Simple to implement.
- Easy to maintain.
- Faster than scanning the entire data file.
- Suitable for small and medium-sized databases.
- Requires fewer index levels.

Limitations

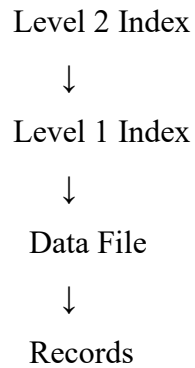
- Index can become very large for huge tables.
- Searching a large index can require multiple block accesses.
- Performance decreases when the index itself becomes too large.

3. Multi-Level Indexing

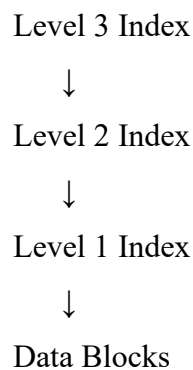
In multi-level indexing, an index is created over another index.

Instead of searching a very large index directly, the database searches the higher-level index first, then moves down to lower levels.

Structure



For a large database:



Advantages

- Efficient for very large databases.
- Reduces the number of disk accesses.
- Higher-level indexes are much smaller.
- Searching remains efficient even when the data file is huge.
- Forms the basic idea behind tree-based indexes.

Limitations

- More complex than single-level indexing.
- Requires additional storage for multiple index levels.
- Insertions and deletions may require updates at several levels.
- More difficult to maintain manually.

4. Comparison

Feature	Single-Level Indexing	Multi-Level Indexing
Number of levels	One	Multiple
Structure	Index → Data	Index → Index → Data
Complexity	Simple	More complex
Search speed	Fast	Faster for very large indexes
Memory usage	Lower initially	More index storage
Disk accesses	Can increase for large indexes	Reduced
Maintenance	Easy	More complex
Small databases	Suitable	Usually unnecessary
Large databases	Less efficient	Highly suitable
Scalability	Limited	High

5. Retrieval Efficiency

Single-level indexing

Query

↓

Large Index

↓

Data Block

↓

Record

Multi-level indexing

Query

↓

Level 3

↓

Level 2

↓

Level 1

↓

Data Block

↓

Record

Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO SingleLevelIndex VALUES

-> (101,'Ravi',85),
-> (105,'Kiran',74),
-> (110,'Arun',91),
-> (115,'Priya',78),
-> (120,'Meena',88),
-> (125,'Rahul',95),
-> (130,'Sneha',82);

Query OK, 7 rows affected (0.01 sec)

Records: 7 Duplicates: 0 Warnings: 0

mysql> INSERT INTO MultiLevelIndex VALUES

-> (101,'Ravi',85),
-> (105,'Kiran',74),
-> (110,'Arun',91),
-> (115,'Priya',78),
-> (120,'Meena',88),
-> (125,'Rahul',95),
-> (130,'Sneha',82);

Query OK, 7 rows affected (0.02 sec)

Records: 7 Duplicates: 0 Warnings: 0

mysql> SELECT *

-> FROM SingleLevelIndex
-> ORDER BY Student_ID;

Student_ID	Student_Name	Marks
101	Ravi	85
105	Kiran	74
110	Arun	91
115	Priya	78
120	Meena	88
125	Rahul	95
130	Sneha	82

7 rows in set (0.00 sec)

mysql> SELECT *

-> FROM MultiLevelIndex
-> ORDER BY Student_ID;

Student_ID	Student_Name	Marks
101	Ravi	85
105	Kiran	74
110	Arun	91
115	Priya	78
120	Meena	88
125	Rahul	95
130	Sneha	82

7 rows in set (0.00 sec)

mysql> SELECT *

-> FROM SingleLevelIndex
-> WHERE Student_ID = 120;

Student_ID	Student_Name	Marks
120	Meena	88

1 row in set (0.00 sec)

```
mysql> SELECT *
-> FROM MultiLevelIndex
-> WHERE Student_ID BETWEEN 110 AND 125
-> ORDER BY Student_ID;
```

Student_ID	Student_Name	Marks
110	Arun	91
115	Priya	78
120	Meena	88
125	Rahul	95

4 rows in set (0.00 sec)

```
mysql> SELECT
-> 'Single-Level Index' AS Index_Type,
-> 'One index level directly points to data records' AS Structure,
-> 'Fast for moderate-sized files' AS Retrieval_Performance
-> UNION ALL
-> SELECT
-> 'Multi-Level Index',
-> 'Multiple index levels reduce the search space',
-> 'More efficient for large databases';
```

Index_Type	Structure	Retrieval_Performance
Single-Level Index	One index level directly points to data records	Fast for moderate-sized files
Multi-Level Index	Multiple index levels reduce the search space	More efficient for large databases

2 rows in set (0.00 sec)

```
mysql> SELECT
-> 'Single-Level Index' AS Index_Type,
-> 'Simple structure and easy maintenance' AS Advantage,
-> 'Less efficient when the index becomes large' AS Limitation
-> UNION ALL
-> SELECT
-> 'Multi-Level Index',
-> 'Faster searching for large indexes',
-> 'Requires additional index storage and maintenance';
```

Index_Type	Advantage	Limitation
Single-Level Index	Simple structure and easy maintenance	Less efficient when the index becomes large
Multi-Level Index	Faster searching for large indexes	Requires additional index storage and maintenance

2 rows in set (0.00 sec)

```
mysql> SELECT
-> 'Single-Level Index' AS Index_Type,
-> 'Lower' AS Memory_Requirement,
-> 'Fast for small/moderate indexes' AS Access_Speed,
-> 'Simple database retrieval' AS Suitability
-> UNION ALL
-> SELECT
-> 'Multi-Level Index',
-> 'Higher',
-> 'Very efficient for large indexes',
-> 'Large-scale database retrieval';
```

Index_Type	Memory_Requirement	Access_Speed	Suitability
Single-Level Index	Lower	Fast for small/moderate indexes	Simple database retrieval
Multi-Level Index	Higher	Very efficient for large indexes	Large-scale database retrieval

2 rows in set (0.00 sec)

10Q) Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases

ANS

1. Theory

In database systems, hashing and indexing are used to improve data retrieval performance.

The best technique depends on the type of query:

- **Exact-match query:** WHERE Student_ID = 105
- **Range query:** WHERE Student_ID BETWEEN 100 AND 200

In general:

Hashing is excellent for exact-match queries.

B+ Tree indexing is excellent for both exact-match and range queries.

2. Hashing Technique

Hashing uses a hash function to map a search key to a bucket.

For example:

$$h(\text{Student_ID}) = \text{Student_ID} \text{ MOD } 5$$

For Student ID 105:

$$105 \text{ MOD } 5 = 0$$

Therefore, the record is stored in **Bucket 0**.

Structure

Student_ID



Hash Function



Bucket



Record

Exact-Match Query

Example:

SELECT *

FROM Student

WHERE Student_ID = 105;

Hashing can directly calculate the bucket containing 105.

Range Query

Example:

SELECT *

FROM Student

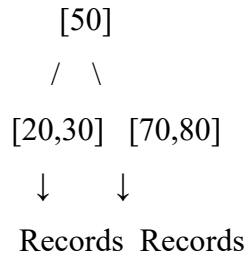
WHERE Student_ID BETWEEN 100 AND 200;

Hashing is not efficient because values within the range may be distributed across many unrelated buckets.

3. Indexing Technique

A database index provides an additional access structure for finding records efficiently.

For example, a B+ Tree index:



Exact-Match Query

SELECT *

FROM Student

WHERE Student_ID = 105;

The index quickly navigates to the required key.

Range Query

SELECT *

FROM Student

WHERE Student_ID BETWEEN 100 AND 200;

A B+ Tree can find the first key and then traverse the ordered leaf entries.

Therefore, indexing is highly suitable for range queries.

4. Hashing vs Indexing

Feature	Hashing	B+ Tree Indexing
Exact-match search	Excellent	Excellent
Range search	Poor	Excellent
Search complexity	Average O(1)	O(log n)
Data ordering	Not maintained	Maintained
Equality queries	Best suited	Good
Range queries	Not suitable	Best suited
Sequential access	Poor	Good
Collision handling	Required	Not applicable
Storage overhead	Bucket/overflow storage	Index storage
Scalability	Good	Excellent
Typical database use	Hash-based access	Very common

5. Performance for Exact-Match Queries

Suppose there are **1,000,000 records**.

Hashing

Student_ID = 750000



Hash function



Bucket



Record

Average lookup is approximately:

O(1)

provided the hash function distributes keys well and collisions are limited.

B+ Tree

Student_ID = 750000



Root



Internal Node



Leaf



Record

Search is approximately: $O(\log n)$

```

mysql>
mysql> CREATE TABLE IndexPerformance (
  ->     Student_ID INT PRIMARY KEY,
  ->     Student_Name VARCHAR(100),
  ->     Department VARCHAR(50),
  ->     Marks INT
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO HashPerformance VALUES
  -> (101,'Ravi','CSE',85),
  -> (105,'Kiran','ECE',74),
  -> (110,'Arun','IT',91),
  -> (115,'Priya','CSE',78),
  -> (120,'Meena','ECE',88),
  -> (125,'Rahul','IT',95),
  -> (130,'Sneha','CSE',82);
Query OK, 7 rows affected (0.01 sec)
Records: 7 Duplicates: 0 Warnings: 0

mysql> INSERT INTO IndexPerformance VALUES
  -> (101,'Ravi','CSE',85),
  -> (105,'Kiran','ECE',74),
  -> (110,'Arun','IT',91),
  -> (115,'Priya','CSE',78),
  -> (120,'Meena','ECE',88),
  -> (125,'Rahul','IT',95),
  -> (130,'Sneha','CSE',82);
Query OK, 7 rows affected (0.05 sec)
Records: 7 Duplicates: 0 Warnings: 0

mysql> SELECT *
  -> FROM HashPerformance
  -> WHERE Student_ID = 120;
+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | Marks |
+-----+-----+-----+-----+
|         120 | Meena        | ECE        |    88 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT *
  -> FROM IndexPerformance
  -> WHERE Student_ID = 120;
+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | Marks |
+-----+-----+-----+-----+
|         120 | Meena        | ECE        |    88 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT *
  -> FROM HashPerformance
  -> WHERE Student_ID BETWEEN 110 AND 125
  -> ORDER BY Student_ID;
+-----+-----+-----+-----+
| Student_ID | Student_Name | Department | Marks |
+-----+-----+-----+-----+
|         110 | Arun         | IT         |    91 |
|         115 | Priya        | CSE        |    78 |
|         120 | Meena        | ECE        |    88 |
|         125 | Rahul        | IT         |    95 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

```

```
mysql> SELECT *
-> FROM IndexPerformance
-> WHERE Student_ID BETWEEN 110 AND 125
-> ORDER BY Student_ID;
```

Student_ID	Student_Name	Department	Marks
110	Arun	IT	91
115	Priya	CSE	78
120	Meena	ECE	88
125	Rahul	IT	95

4 rows in set (0.00 sec)

```
mysql> SELECT
-> 'Hashing' AS Technique,
-> 'Very fast for exact-match queries' AS Exact_Match,
-> 'Less suitable for range queries' AS Range_Query
-> UNION ALL
-> SELECT
-> 'Indexing',
-> 'Fast for exact-match queries',
-> 'Highly efficient for range queries';
```

Technique	Exact_Match	Range_Query
Hashing	Very fast for exact-match queries	Less suitable for range queries
Indexing	Fast for exact-match queries	Highly efficient for range queries

2 rows in set (0.00 sec)

```
mysql> SELECT
-> 'Hashing' AS Technique,
-> 'Direct bucket-based access' AS Advantage,
-> 'Poor range-query performance' AS Limitation
-> UNION ALL
-> SELECT
-> 'Indexing',
-> 'Supports exact-match and range queries',
-> 'Requires additional index storage';
```

Technique	Advantage	Limitation
Hashing	Direct bucket-based access	Poor range-query performance
Indexing	Supports exact-match and range queries	Requires additional index storage

2 rows in set (0.00 sec)

```
mysql> SELECT
-> 'Hashing' AS Technique,
-> 'Excellent' AS Exact_Match_Performance,
-> 'Low' AS Range_Query_Performance,
-> 'Best suited for exact-match retrieval' AS Recommendation
-> UNION ALL
-> SELECT
-> 'Indexing',
-> 'Very Good',
-> 'Excellent',
-> 'Best suited for exact-match and range retrieval';
```

Technique	Exact_Match_Performance	Range_Query_Performance	Recommendation
Hashing	Excellent	Low	Best suited for exact-match retrieval
Indexing	Very Good	Excellent	Best suited for exact-match and range retrieval

2 rows in set (0.00 sec)